

Technical Report

Embedded System Project:
Advanced PC FAN speed controller
with over-speed alarm.

Author: Martins Osemudiamen
Okoh.

TU Dublin ID: A00047170

Table of Content

I. Alm	1
II. Objective	1
III. Design	1
IV. Components	2
V. Software Code	9
VI. Testing	20
VII. Conclusion	24
VIII. Reference	25

Table of Table

I.	Table 1: STM32L432KC	2
II.	Table 2: PIN connection	2
III.	Table 3: Button	3
IV.	Table 4: LED	3
V.	Table 5: Resistor	3
VI.	Table 6: Rotary Encoder	4
VII.	Table 7: PIN connection	4
VIII.	Table 8: LCD Screen	5
IX.	Table 9: PIN connection	5
X.	Table 10: Buzzer	5
XI.	Table 11: PC FAN	6
XII.	Table 12: 4-wire PinOut	6
XIII.	Table 13: USB Cable	7
XIV.	Table 14: Breadboard	7
XV.	Table 15: Connecting wires	7
XVI.	Table 16: Power Supply	8

Tables of Figures

I.	<i>Figure 1: Circuit Design of PC fan controller using KICad 10.0.</i>	1
II.	<i>Figure 2: Declaration of functions and addition of necessary files and libraries.</i>	9
III.	<i>Figure 3: Declaration of global variables.</i>	9
IV.	<i>Figure 4: Main function of the software implementation part 1.</i>	10
V.	<i>Figure 5: Main function of the software implementation part 2.</i>	11
VI.	<i>Figure 6: Main function of the software implementation part 3.</i>	11
VII.	<i>Figure 7: Main function of the software implementation part 4.</i>	12
VIII.	<i>Figure 8: Setup Function.</i>	13
IX.	<i>Figure 9: delay , Pull Up resistor and Pinmode.</i>	14
X.	<i>Figure 10: Timer 2.</i>	15
XI.	<i>Figure 11: Timer 16.</i>	16
XII.	<i>Figure 12: Timer 6.</i>	16
XIII.	<i>Figure 13: Timer 6 interrupt .</i>	17
XIV.	<i>Figure 14: Function of installing delay.</i>	17
XV.	<i>Figure 15: Function to enable pull-up resistor.</i>	17
XVI.	<i>Figure 16: Function for PIN mode selection.</i>	18
XVII.	<i>Figure 17: Function for initializing clock.</i>	18
XVIII.	<i>Figure 18: Function of initializing serial communication and pin definition.</i>	18
XIX.	<i>Figure 19: Function for writing via serial communication.</i>	19
XX.	<i>Figure 20: Function for write via serial communication to terminal display.</i>	19
XXI.	<i>Figure 21: Function to handle interrupt to counting the speed pulse.</i>	19
XXII.	<i>Figure 22: Function for milliseconds counting and delays.</i>	19
XXIII.	<i>Figure 23: Hardware implementation of bread board.</i>	20
XXIV.	<i>Figure 24: Power supply.</i>	21
XXV.	<i>Figure 25: Output result 1 on terminal.</i>	21
XXVI.	<i>Figure 26: Output result 2 on terminal.</i>	21
XXVII.	<i>Figure 27: Fan output speed signal 1 viewed on oscilloscope.</i>	22
XXVIII.	<i>Figure 28: PC fan Assemble with LED ON and Screen.</i>	22
XXIX.	<i>Figure 29: Output result 2 on terminal.</i>	23
XXX.	<i>Figure 30: Fan output speed signal 2 viewed on oscilloscope.</i>	23
XXXI.	<i>Figure 31: PC fan Assemble with LED OFF, Buzzer toggling and Screen .</i>	24

Components:

1) STM32L432KC Microcontroller:

Table 1: STM32L432KC

Specification	Detail
Manufacturer	STMicroelectronics
Core	ARM Cortex-M4F (with FPU)
Max frequency	80 MHz
Flash memory	256 KB
SRAM	64 KB
Operating voltage	1.71V – 3.6V
Package	32-pin QFN
Key peripherals	GPIO, TIM2, TIM6, TIM16, USART, SPI, EXTI, NVIC

Component Role: Central processing unit. Runs the main control loop, reads encoder and push button inputs, generates PWM for the fan, controls the LED and buzzer, communicates with the LCD via SPI, and outputs debug data over USB serial. It also serve as a 3.3 volt power supply for the circuit [1].

Connection: The Pin connection is as in Table 2.

Table 2: PIN connection

Pin	Direction	Function
PA0	Input	TIM2_CH1 – Encoder CLK (AF1)
PA2	Output	USART2_TX – Serial transmit (AF7)
PA5	Output	SPI1_SCK – Display clock
PA6	Output	TIM16_CH1 – PWM fan control (AF14)
PA7	Output	SPI1_MOSI – Display data
PA15	Input	USART2_RX – Serial receive (AF3)
PB3	Input	TIM2_CH2 – Encoder DT (AF1)
PB4	Input	EXTI4 – Tachometer pulse input (falling edge)
PB5	Output	GPIO – LED overspeed indicator
PB6	Output	GPIO – Buzzer tone output (toggled by TIM6)

All peripherals are clocked through the internal PLL configured to 80 MHz at startup in `initClocks()`. `SysTick` is configured to interrupt every 1 ms to provide the `delay_ms()` timebase [2].

2) Push Down Button:

Table 3: Button

Specification	Detail
Type	Tactile momentary switch
Number of pins	2 connected internally
Actuation Force	100 - 300gf
Contact resistance	< 100 mΩ
Operating life	10,000 – 100,000 cycles

Component Role: Provides user input for controlling the system (e.g., starting/stopping the fan, resetting the system, or toggling display modes). In this project, it may serve as the tachometer pulse input or a manual override switch [3].

Connection: Connected to PB4 with internal pull-up enabled. When pressed, the pin reads LOW (0V); when released, pull-up keeps it HIGH (3.3V). Falling edge triggers an external interrupt.

3) Light Emitting Diode (LED) :

Table 4: LED

Specification	Detail
Type	Standard 5mm or SMD LED
Color	Green
Forward voltage (Vf)	2.8V – 3.2V (green)
Forward current (If)	10 – 20 mA
Viewing angle	30° – 60°
Luminous intensity	1000 – 5000 mcd

Component Role: Visual status indicator. Illuminates when the fan speed is within safe limits (≤ 3500 RPM) and turns off during over speed conditions [4].

Connection: Anode connected to PB5 through a 330Ω current-limiting resistor; cathode connected to ground.

4) 330 Ohm resistor:

Table 5: Resistor

Specification	Detail
Resistance	330 Ω $\pm 5\%$
Power rating	0.25W (1/4W)
Type	Carbon film or metal film
Color code	Orange – Orange – Brown – Gold

Component Role: Current-limiting resistor for the LED. 330Ω provides approximately 4 mA current, ensuring safe operation without exceeding the LED's maximum rating or the microcontroller's GPIO current limit (20 mA per pin) [5].

Connection: Placed in series between PB5 (microcontroller output) and the LED anode.

5) Rotary Encoder:

Table 6: Rotary Encoder

Specification	Detail
Type	Incremental quadrature encoder
Pulses per revolution (PPR)	2 (as defined in code: <code>#define pulse_per_rev 2</code>)
Output signals	Channel A (CLK), Channel B (DT)
Output type	Open-collector or push-pull
Supply voltage	3.3V – 5V
Mechanical life	30,000 – 100,000 cycles
Encoder Count Range	0 – 159 (set by TIM2 ARR = 159 in code)

Component Role: Provides the primary user input for fan speed control. As the knob is rotated clockwise or counter-clockwise, it generates two out-of-phase square wave signals (CLK and DT) which TIM2 decodes in hardware encoder mode 3 (quadrature). The resulting count value in TIM2->CNT (ranging 0–159) is read in the main loop and scaled to a PWM duty cycle on TIM16, directly controlling fan speed. Rotating fully clockwise produces maximum PWM (100% duty), and fully counter-clockwise produces minimum (0%) [6].

Connection: The Pin connection is as in Table 2.

Table 7: PIN connection

Encoder Pin	STM32L432 Pin
CLK	PA0
DT	PB3
VCC	3.3v
GND	GND
SW	-

How it works: PA0 and PB3 are configured in alternate function mode (AF1) to route directly into TIM2's hardware encoder interface. TIM2 is configured in encoder mode 3 (TIM2->SMCR = 3), meaning both edges of both channels are counted, giving smooth and accurate position tracking. The counter wraps between 0 and 159 as set by TIM2->ARR = 159. No software debouncing is required as the hardware timer filters signal noise inherently. Internal pull-ups are not explicitly set for PA0/PB3 as the encoder module typically includes its own pull-up resistors on CLK and DT.

6) LCD Screen (SPI Interface)

Table 8: LCD Screen

Specification	Detail
Type	1.8" TFT SPI LCD v1.1
Resolution	128 × 160 pixels
Display Driver IC	ST7735S
Interface	SPI (Serial Peripheral Interface)
Operating voltage	3.3V
Colour Depth	16-bit (65,536 colours)
Viewable Area	1.8 inches diagonal
Pixel Format	RGB565

Component Role: Provides a real-time visual readout of system state to the user. In this project, it displays the current rotary encoder position (pos) and the calculated fan speed (RPM). Text is rendered using `printText()` and screen regions are cleared between updates using `fillRectangle()` to prevent display artefacts. Colour coding is used to distinguish data fields — yellow for encoder position, green for RPM [7].

Connection: The Pin connection is as in Table 9.

Table 9: PIN connection

Display Pin	STM32L432 Pin
VCC	3.3v
GND	GND
SCL	PA5
SDA	PA7
RESET	PB0
DC	PB1
CS	PA4
LED	3.3v

7) Buzzer:

Table 10: Buzzer

Specification	Detail
Type	Piezoelectric transducer
Resonant frequency	500 Hz (typical)
Operating voltage	3V – 12V
Current consumption	5 – 15 mA
Sound pressure level	75 – 85 dB at 10 cm

Component Role: Audible alarm. Activates when the fan speed exceeds 3500 RPM, producing a 500 Hz tone to alert the user of an overspeed condition [8].

Connection: Positive (+) to PB6 (microcontroller output) and Negative (-) to Ground

Drive method: Timer 6 generates a 1000 Hz interrupt that toggles PB6 at 500 Hz when playing = 1, creating a square wave. The buzzer produces sound due to the piezoelectric element vibrating at this frequency .

Behavior:

Speed $\leq$ 3500 RPM: playing = 0 via PB6 LOW I.e silent

Speed > 3500 RPM: playing = 1 via PB6 toggles at 500 Hz I.e audible tone

7) PC FAN with 4-wire connection:

Table 11: PC FAN

Specification	Detail
Type	Brushless DC fan (computer cooling fan)
Wire colors (standard)	Black (GND), Yellow (12V), Green (Tachometer), Blue (PWM control)
Supply voltage	12V DC
Current draw	0.1A – 0.5A (typical)
Speed range	0 – 4000+ RPM

Table 12: 4-wire PinOut

Wire Color	Signal	Function
Black	GND	Ground (0V)
Yellow	+12V	Power supply (12V)
Green	Tachometer	Open-collector output (2 pulses per revolution)
Blue	PWM	Speed control input (25 kHz, 0–100% duty cycle)

Component Role: The main actuator being controlled and monitored. The microcontroller sends a PWM signal to the blue wire to regulate fan speed, and reads tachometer pulses from the green wire to measure actual RPM [9].

Connection:Black (GND) to Common ground, Yellow (12V) to External 12V power supply, Green (Tachometer) to PB4 (EXTI4 interrupt input) – counts speed pulses and Blue (PWM) to PA6 (TIM16 CH1) – receives 25 kHz PWM signal.

How speed is measured: The fan's tachometer output produces 2 pulses per revolution (open-collector). Each falling edge on PB4 triggers an interrupt, incrementing `pulse_count`. The main loop calculates speed every 0.1333 seconds.

How speed is controlled: The PWM duty cycle on PA6 determines fan speed. The encoder value (0–159) is scaled to set `TIM16->CCR1` (0–3199).

Duty cycle range: 0% (fan off) to 100% (maximum speed).

8) USB Cable (serial communication):

Table 13: USB Cable

Specification	Detail
Type	USB C
Length	0.5m – 1m (typical)
Interface	USB 2.0 full-speed
Purpose	Provides serial communication between STM32 and PC

Component Role: Enables real-time monitoring and debugging. The STM32L432's USART peripheral is connected to the USB port via an onboard USB-to-serial converter (on Nucleo boards or external FTDI adapter). The PC receives debug messages at 9600 baud using a serial terminal (PuTTY, screen, minicom) [10]. Data transmitted are Initialization verification (register values) and Runtime status (encoder value, PWM duty, RPM, buzzer state).

Connection: USB cable connects the microcontroller board to a computer. The computer's serial monitor displays the debug output.

9) Breadboard:

Table 14: Breadboard

Specification	Detail
Type	Solderless breadboard
Size	400 – 830 tie points (typical)
Contact material	Nickel-plated or tin-plated phosphor bronze
Pitch	2.54mm (0.1 inch)

Component Role: Prototyping platform for assembling the circuit without soldering. Components and wires are inserted into the breadboard's contact holes, which are electrically connected in rows and columns. The ground row is used as neural ground connections [11].

10) Connecting wires:

Table 15: Connecting wires

Specification	Detail
Type	Jumper wires
Gauge	22 AWG – 26 AWG
Length	10 cm – 30 cm
Connector type	Dupont (2.54mm pitch)

Component Role: Establish all electrical connections between components on the breadboard [12].

Typical connections include:

- i. Microcontroller pins to breadboard power rails
- ii. Encoder to microcontroller pins (PA0, PB3, VCC, GND)
- iii. Push button to PB4 and ground
- iv. LED and resistor to PB5 and ground
- v. Buzzer to PB6 and ground
- vi. LCD to SPI pins, VCC, and GND
- vii. Fan tachometer (green) to PB4
- viii. Fan PWM (blue) to PA6
- ix. Fan ground (black) to common ground
- x. Fan 12V (yellow) to 12V supply
- xi. 3.3V and GND to breadboard power rails

11) Power Supply:

Table 16: Power Supply

Specification	Detail
Type	Regulated DC Bench Power Supply
Output Voltage	0 – 30V DC (adjustable)
Output Current	0 – 5A (adjustable limit)
Input Voltage	220–240V AC, 50Hz
Output Terminals	Positive (+), Negative (–), Ground (GND)
Accuracy	±(0.5% to 1%) depending on model
Ripple/Noise	Typically < 10mV

Component Role: The positive terminal (+12V) of the power supply is connected to the fan's power input wire, while the negative terminal (GND) is connected to the fan ground wire I.e +terminal to Fan VCC (12v) and -terminal to GND [13].

Source Code:

```
#include <stm32l432xx.h>
#include <stdint.h>
#include <math.h>
// added from serial file
#include <eeng1030_lib.h>
#include <stdio.h>
#include <errno.h>
#include <sys/unistd.h> // STDOUT_FILENO, STDERR_FILENO
#include "display.h"

void setup(void);
void delay(volatile uint32_t dly);
void enablePullUp(GPIO_TypeDef *Port, uint32_t BitNumber);
void pinMode(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t Mode);
void initADC(void);
int readADC(int chan);
void initTimer2(void);
void setTimer2Duty(int duty);
void initClocks(void);
void delay_ms(unsigned dly);
void initEncoder(void);
void initBuzzerTimer(void);
void initSerial(uint32_t baudrate);
void putc(char c);
```

Figure 2: Declaration of functions and addition of necessary files and libraries.

Figure 2 above shows a hardware initialization and utility library for an STM32L432xx microcontroller (ARM Cortex-M4). It provides core functions for peripheral control, timing, analog reading, PWM generation, encoder interfacing, buzzer control, and serial communication. The code appears to be part of a larger embedded systems project (likely for a robotics or mechatronics course, given the eeng1030_lib.h reference).

```
// define needed constants
#define pulse_per_rev 2
#define BEEP_SAMPLES 4790
int16_t audioBuffer[BEEP_SAMPLES * 2];
volatile uint8_t playing = 0; // 0 = buzzer off, 1 = buzzer on (controlled by TIM6)

int vin, speed_hz, speed_rpm;
volatile unsigned pulse_count, milliseconds;
```

Figure 3: Declaration of global variables.

Figure 3 above shows the constants and variables for motor speed sensing and buzzer audio playback on an STM32 microcontroller. The constant pulse_per_rev is set to 2, meaning the motor encoder generates two pulses per revolution, while BEEP_SAMPLES defines the audio buffer size as 4790 samples. An int16_t array called audioBuffer allocates twice that many elements (9580 total) to store audio data for buzzer tones, and a volatile uint8_t flag named playing tracks whether the buzzer is active (1) or off (0), with updates controlled by Timer 6

interrupts. Integer variables `vin`, `speed_hz`, and `speed_rpm` hold motor voltage input and speed values in hertz and RPM respectively. Finally, two volatile unsigned variables—`pulse_count` for counting encoder pulses and milliseconds for system timing—are declared volatile because they are modified inside interrupt service routines but accessed in the main program loop.

```
int main()
{
    setup();
    SysTick->LOAD = 80000-1; // SysTick clock = 80MHz. 80000000/80000=1000
    SysTick->CTRL = 7; // enable systick counter and its interrupts
    SysTick->VAL = 10; // start from a low number so we don't wait for ages for first interrupt

    initEncoder(); // TIM2 (input)
    init_display(); // SPI display
    initTimer16(); //TIM16 (PWM)
    initBuzzerTimer(); // TIM6 - buzzer tone generator

    delay_ms(2000);

    printf("MODER PA6: %lu\r\n", (GPIOA->MODER >> 12) & 0x3); // should be 2
    printf("AFR PA6: %lu\r\n", (GPIOA->AFR[0] >> 24) & 0xF); // should be 14
    printf("TIM16 CR1: %lu\r\n", TIM16->CR1); // should be 0x81
    printf("TIM16 CCER: %lu\r\n", TIM16->CCER); // should be 0x1
    printf("TIM16 BDTR: %lu\r\n", TIM16->BDTR); // should be 0x8000
    printf("TIM16 ARR: %lu\r\n", TIM16->ARR); // should be 3199
    printf("TIM16 CCR1: %lu\r\n", TIM16->CCR1); // should be 1600
}
```

Figure 4: Main function of the software implementation part 1.

This `main()` function in Figure 4 above initializes the entire embedded system and then verifies the configuration by printing register values. It begins by calling `setup()` for basic hardware initialization, then configures the SysTick timer to generate interrupts every 1 millisecond (since an 80 MHz clock with a load value of 80,000 produces exactly 1000 Hz). It initializes the encoder on Timer 2, an SPI display, Timer 16 for PWM output, and Timer 6 for buzzer tone generation. After a 2-second delay to allow everything to stabilize, it prints the current register values for GPIO pin PA6 (checking its mode and alternate function), followed by several Timer 16 registers including CR1 (control), CCER (capture/compare enable), BDTR (break and dead-time), ARR (auto-reload value), and CCR1 (capture/compare value). These print statements serve as diagnostic checks to confirm that Timer 16 was configured correctly—for example, the ARR should be 3199 and CCR1 should be 1600, indicating a 50% duty cycle on the PWM output.

```

while(1)
{
    vin = TIM2->CNT;

    TIM16->CCR1 = (vin * 3199) / 159;

    //uncomment
    uint32_t pulses;

    __disable_irq();
    pulses = pulse_count;

    pulse_count = 0;

    //uncomment
    __enable_irq();

    // evaluate speed
    speed_hz = (pulses / pulse_per_rev);
    speed_rpm = (speed_hz * 450);
}

```

Figure 5: Main function of the software implementation part 2.

Figure 5 above shows continuous loop reads the motor's input voltage from Timer 2's counter register, scales it by a factor of 3199/159 (approximately 20.12) and writes the result to Timer 16's capture compare register to control a PWM output—likely for motor speed or LED brightness. It then safely copies the volatile pulse_count variable into a local pulses variable by disabling interrupts briefly to prevent corruption during the read, resets pulse_count to zero for the next measurement interval, and re-enables interrupts. Finally, it calculates the motor speed in hertz by dividing the accumulated pulses by pulse_per_rev (which is 2), then converts that to RPM by multiplying by 450—the conversion factor assumes the measurement period is 0.1333 seconds (since 60 seconds divided by 0.1333 equals 450). This loop continuously reads feedback, updates control outputs, and computes rotational speed in real time.

```

// LED: ON when speed is safe (<=3500), OFF when too fast
if (speed_rpm <= 3500)
{
    GPIOB->ODR |= (1 << 5); // turn LED on – normal speed
    playing = 0;             // buzzer off
}
else
{
    GPIOB->ODR &= ~(1 << 5); // turn LED off – overspeed
    playing = 1;             // buzzer on via TIM6
}

```

Figure 6: Main function of the software implementation part 3.

This code implemented in Figure 6 shows a safety monitoring system that controls an LED and buzzer based on motor speed. If `speed_rpm` is 3500 RPM or less (safe zone), it turns the LED on by setting bit 5 of GPIOB's output data register, and ensures the buzzer is off by setting `playing = 0`. If the speed exceeds 3500 RPM (overspeed condition), it turns the LED off by clearing bit 5, and activates the buzzer by setting `playing = 1`, which enables Timer 6 to generate an audible alarm tone. This provides both visual feedback (LED on/off) and audible warning (buzzer) to alert the user when the motor is running too fast.

```
// print fan speed
printf("ENC: %d, CCR1: %lu, RPM: %d,buzz: %lu\r\n", vin, TIM16->CCR1, speed_rpm,(GPIOB->ODR >> 6) & 1);

// Clear text areas for display
fillRectangle(0,10,160,20,RGBToWord(0,0,0));
fillRectangle(0,30,160,20,RGBToWord(0,0,0));

// Display values
char buffer[50];

sprintf(buffer, "pos: %3d", vin);
printText(buffer, 5, 10, RGBToWord(255,255,0), 0);

sprintf(buffer, "RPM: %4d", speed_rpm);
printText(buffer, 5, 30, RGBToWord(0,255,0), 0);

delay_ms(100);
```

Figure 7: Main function of the software implementation part 4.

This code in Figure 7 handles real-time data output and user interface updates. It first prints a status line over serial containing the encoder value (`vin`), the current PWM capture compare register value from Timer 16, the calculated motor speed in RPM, and the buzzer state (extracted from bit 6 of GPIOB's output data register). It then updates an SPI display by clearing two text areas (at Y-positions 10 and 20, and 30 and 40) using black rectangles. The encoder position (`vin`) is formatted into a string labeled "pos" and displayed in yellow text at the top, while the RPM value is formatted and displayed in green text below. Finally, a 100 millisecond delay sets the loop's update rate to approximately 10 Hz, providing stable, readable updates without overwhelming the system.

```

void setup(void)
{
    initClocks();
    RCC->AHB2ENR |= (1 << 0) | (1 << 1); // turn on GPIOA and GPIOB
    pinMode(GPIOB,3,1); // PB3 digital output
    pinMode(GPIOB,4,0); // PB4 digital input (tachometer)
    pinMode(GPIOB, 5, 1); // PB5 output for LED
    pinMode(GPIOB, 6, 1); // PB6 as output for buzzer
    enablePullUp(GPIOB,4); // pull-up for tach input
    pinMode(GPIOA,3,2); // alternative function mode

    GPIOA->AFR[0] &= ~(3 << (2*3)); // Clear out old alternative function bits
    GPIOA->AFR[0] |= 1 << (4*3); // Select alternative function 1

    // serial communication
    initSerial(9600); //115200

    // EXTI4 interrupt for tach pulse counting on PB4
    RCC->APB2ENR |= (1 << 0); // enable SYSCFG
    SYSCFG->EXTICR[1] &= ~(7 << 0); // clear perhaps previously set bits
    SYSCFG->EXTICR[1] |= (1 << 0); // map EXTI2 interrupt to PB4
    EXTI->FTSR1 |= (1 << 4); // select falling edge trigger for PB4 input
    EXTI->IMR1 |= (1 << 4); // enable PB4 interrupt
    NVIC->ISER[0] |= (1 << 10); // IRQ 10 maps to EXTI4
    __enable_irq();
}

```

Figure 8: Setup Function.

This `setup()` function shown in Figure 8 above performs all low-level hardware initialization for the system. It starts by calling `initClocks()` to configure system clocks, then enables GPIO ports A and B via the AHB2ENR register. It configures several GPIO pins: PB3 as a digital output, PB4 as a digital input for tachometer sensing (with a pull-up enabled), PB5 as an output for the status LED, and PB6 as an output for the buzzer. Pin PA3 is set to alternative function mode, and its alternate function is configured to AF1 (likely for timer input capture). Serial communication is initialized at 9600 baud for debugging output. The function then sets up an external interrupt on PB4 to count tachometer pulses: it enables the SYSCFG clock, configures the EXTI to map PB4 to interrupt line 4, selects falling edge triggering, enables the interrupt in the EXTI controller, and enables the EXTI4 interrupt in the NVIC (IRQ 10). Finally, it globally enables interrupts with `__enable_irq()`, allowing the tachometer pulse counting to begin.


```

void delay(volatile uint32_t dly)
{
    while(dly--);
}

void enablePullUp(GPIO_TypeDef *Port, uint32_t BitNumber)
{
    Port->PUPDR = Port->PUPDR & ~(3u << BitNumber*2); // clear pull-up resistor bits
    Port->PUPDR = Port->PUPDR | (1u << BitNumber*2); // set pull-up bit
}

void pinMode(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t Mode)
{
    /*
    Modes : 00 = input
           01 = output
           10 = special function
           11 = analog mode
    */
    uint32_t mode_value = Port->MODER;
    Mode = Mode << (2 * BitNumber);
    mode_value = mode_value & ~(3u << (BitNumber * 2));
    mode_value = mode_value | Mode;
    Port->MODER = mode_value;
}

```

Figure 9: delay, Pull Up resistor and Pinmode.

These three functions shown in Figure 9 above provide basic GPIO control and timing utilities.

`delay(volatile uint32_t dly)` creates a simple blocking delay by decrementing the input value in a tight while loop until it reaches zero. The parameter is declared `volatile` to prevent the compiler from optimizing away the loop.

`enablePullUp(GPIO_TypeDef *Port, uint32_t BitNumber)` activates the internal pull-up resistor on a specified GPIO pin. It clears the two bits controlling the pull configuration for that pin by ANDing with a mask, then sets the lower bit (binary 01) to select pull-up mode.

`pinMode(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t Mode)` configures the mode of a GPIO pin by writing to the port's MODER register. The Mode parameter (0=input, 1=output, 2=alternate function, 3=analog) is shifted left by twice the bit number to align with the two bits that control that pin, then combined into the register after clearing the existing configuration bits for that pin.

```

// I/O List for Encoder:
// PA0 : TIM2_CH1 (Alternative function 1) : Clock pin on encoder
// PB3 : TIM2_CH2 (Alternative function 1) : DT pin on encoder (quadrature type input)
// PB4 : GPIOA - SW input from encoder (maybe)
void initEncoder()
{
    RCC->AHB2ENR |= (1 << 0) | (1 << 1); // make sure GPIOA,B are turned on
    pinMode(GPIOA,0,2);
    selectAlternateFunction(GPIOA,0,1);
    pinMode(GPIOB,3,2);
    selectAlternateFunction(GPIOB,3,1);
    //pinMode(GPIOB,4,0);
    RCC->APB1ENR1 |= (1 << 0); // enable Timer 2
    TIM2->CR1 = 0;
    TIM2->CNT = 0;
    TIM2->ARR = 159; // full range encoder count

    // Encoder mode 3 (TI1 + TI2)
    TIM2->SMCR = 3;

    // TI1 and TI2 as inputs
    TIM2->CCMR1 = 0;
    TIM2->CCMR1 |= (1 << 0); // CC1S = input
    TIM2->CCMR1 |= (1 << 8); // CC2S = input
    TIM2->CCER = 0;
    TIM2->CR1 |= 1;
}

```

Figure 10: Timer 2.

This `initEncoder()` function configures Timer 2 shown in Figure 10 above in encoder mode to read a quadrature rotary encoder. It enables GPIOA and GPIOB clocks, then configures PA0 as TIM2 channel 1 (clock pin) and PB3 as TIM2 channel 2 (DT pin) both in alternate function mode with AF1 selected. Timer 2 is enabled, its counter is reset to zero, and the auto-reload register ARR is set to 159, which limits the count range from 0 to 159. The SMCR register is set to mode 3, which configures the timer to count on both TI1 and TI2 edges (quadrature decoding). The CCMR1 register sets both capture compare channels 1 and 2 as inputs, and CCER is cleared to configure the inputs. Finally, the timer is started by setting bit 0 in CR1. As the encoder rotates, the timer count automatically increments or decrements based on direction, and the value can be read from TIM2->CNT.

```

47 void initTimer16()
48 {
49     RCC->AHB2ENR |= (1 << 0);    // GPIOA clock on – must be before AFR access
50     RCC->APB2ENR |= (1 << 17);   // TIM16 clock on
51
52     // Manually configure PA6 as AF14 – do NOT use selectAlternateFunction
53     GPIOA->AFR[0] &= ~(0xF << 24); // clear PA6 AF bits (bits 27:24)
54     GPIOA->AFR[0] |= (14 << 24);   // set AF14
55
56     GPIOA->MODER &= ~(3u << 12);   // clear PA6 mode bits
57     GPIOA->MODER |= (2u << 12);    // set alternate function mode
58
59     // CHECK IMMEDIATELY AFTER WRITING
60     printf("MODER inside initTimer16: %lu\r\n", (GPIOA->MODER >> 12) & 0x3);
61
62     TIM16->CR1 = 0;
63     TIM16->PSC = 0;
64     TIM16->ARR = 3199;             // 80MHz / 3200 = 25kHz
65     TIM16->CCMR1 = (0b110 << 4) | (1 << 3) | (1 << 2);
66     TIM16->CCER = (1 << 0);        // CC1E
67     TIM16->CCR1 = 1600;            // 50% duty
68     TIM16->BDTR = (1 << 15);       // MOE
69     TIM16->EGR |= (1 << 0);
70     TIM16->CR1 = (1 << 7) | (1 << 0);
71
72     // CHECK AGAIN AT END OF FUNCTION
73     printf("MODER end of initTimer16: %lu\r\n", (GPIOA->MODER >> 12) & 0x3);
74 }
75 // -----

```

Figure 11: Timer 16.

This `initTimer16()` function shown in Figure 11 configures Timer 16 to generate a PWM signal on pin PA6. It enables the GPIOA clock and Timer 16 clock, then manually configures PA6 to use alternate function 14 (AF14), which is the timer output channel for TIM16. The pin mode is set to alternate function. Diagnostic `printf` statements verify the pin's mode register value immediately after configuration. The timer itself is then set up: prescaler is 0, the auto-reload register ARR is 3199, which with an 80 MHz clock produces a PWM frequency of 25 kHz. The CCMR1 register configures channel 1 for PWM mode 2 (110 binary) with preload enabled, and CCER enables the capture/compare output. CCR1 is set to 1600, giving a 50% duty cycle. The BDTR register's MOE (Main Output Enable) bit is set, which is required for advanced timers to enable PWM output. An update event generates reloads the settings, and CR1 enables auto-reload preload and starts the timer counter.

```

void initBuzzerTimer(void)
{
    RCC->APB1ENR1 |= (1 << 4);    // enable TIM6 clock

    TIM6->CR1 = 0;
    TIM6->PSC = 7999;             // 80MHz / 8000 = 10kHz
    TIM6->ARR = 9;                // 10kHz / 10 = 1kHz toggle → 500Hz buzz tone
    TIM6->DIER = (1 << 0);        // enable update interrupt
    TIM6->EGR = (1 << 0);         // generate update event
    TIM6->CR1 = (1 << 0);         // start timer

    NVIC->ISER[1] |= (1 << 22);   // TIM6 IRQ = 54, bit 22 of ISER[1]
}

```

Figure 12: Timer 6.

This `initBuzzerTimer()` function shown in Figure 12 above configures Timer 6 to generate an interrupt for producing an audible buzzer tone. It enables the TIM6 clock via the APB1ENR1 register, then sets the prescaler to 7999, which divides the 80 MHz system clock down to 10 kHz. The auto-reload register ARR is set to 9, meaning the timer counts from 0 to 9 and then resets, creating an update event at 1 kHz (since 10 kHz divided by 10 equals 1000 Hz). This triggers an interrupt twice per cycle, allowing the buzzer pin to toggle at 500 Hz, which produces an audible 500 Hz tone. The DIER register enables the update interrupt, an update event is generated to load the prescaler and ARR values, and the timer is started by setting bit 0 in CR1. Finally, the interrupt is enabled in the NVIC at IRQ number 54 (bit 22 in ISER[1]), allowing the buzzer control to be handled in the TIM6 interrupt service routine.

```
// -----
// TIM6 ISR - toggles PB6 to generate buzzer tone
// -----
void TIM6_DAC_IRQHandler(void)
{
    TIM6->SR &= ~(1 << 0);           // clear update interrupt flag

    if (playing)
    {
        GPIOB->ODR ^= (1 << 6);      // toggle PB6 → 500Hz tone on buzzer
    }
    else
    {
        GPIOB->ODR &= ~(1 << 6);      // ensure buzzer pin is LOW when off
    }
}
```

Figure 13: Timer 6 interrupt .

This is the interrupt service routine for Timer 6 shown in Figure 13 which generates the buzzer tone. When a timer update interrupt occurs, the handler first clears the interrupt flag by writing 0 to bit 0 of TIM6's status register (SR). It then checks the global playing flag: if playing is 1 (buzzer active), it toggles pin PB6 by XORing bit 6 of GPIOB's output data register, creating a square wave at 500 Hz and producing an audible tone. If playing is 0 (buzzer off), it ensures the buzzer pin is driven low by clearing bit 6, keeping the buzzer silent. This ISR runs at 1000 Hz (every 1 ms), toggling the pin on each interrupt to produce the 500 Hz tone only when the overspeed condition is active.

```
117 void delay(volatile uint32_t dly)
118 {
119     while(dly--);
120 }
```

Figure 14: Function of installing delay.

```
122 void enablePullUp(GPIO_TypeDef *Port, uint32_t BitNumber)
123 {
124     Port->PUPDR = Port->PUPDR & ~(3u << BitNumber*2); // clear pull-up resistor bits
125     Port->PUPDR = Port->PUPDR | (1u << BitNumber*2); // set pull-up bit
126 }
```

Figure 15: Function to enable pull-up resistor.


```

127 void pinMode(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t Mode)
128 {
129     /*
130      * Modes : 00 = input
131      *           01 = output
132      *           10 = special function
133      *           11 = analog mode
134      */
135     uint32_t mode_value = Port->MODER;
136     Mode = Mode << (2 * BitNumber);
137     mode_value = mode_value & ~(3u << (BitNumber * 2));
138     mode_value = mode_value | Mode;
139     Port->MODER = mode_value;
140 }

```

Figure 16: Function for PIN mode selection.

```

158 void initClocks()
159 {
160     // Initialize the clock system to a higher speed.
161     // At boot time, the clock is derived from the MSI clock
162     // which defaults to 4MHz. Will set it to 80MHz
163     // See chapter 6 of the reference manual (RM0393)
164     RCC->CR &= ~(1 << 24); // Make sure PLL is off
165
166     RCC->PLLCFGR = (1 << 25) + (1 << 24) + (1 << 22) + (1 << 21) + (1 << 17) + (80 << 8) + (1 << 0);
167     RCC->CR |= (1 << 24); // Turn PLL on
168     while( (RCC->CR & (1 << 25)) == 0); // Wait for PLL to be ready
169     // configure flash for 4 wait states (required at 80MHz)
170     FLASH->ACR &= ~(1 << 2) + (1 << 1) + (1 << 0);
171     FLASH->ACR |= (1 << 2);
172     RCC->CFGR |= (1 << 1) + (1 << 0); // Select PLL as system clock
173 }

```

Figure 17: Function for initializing clock.

```

175 // added from serial communication file
176 void initSerial(uint32_t baudrate)
177 {
178     RCC->AHB2ENR |= (1 << 0); // make sure GPIOA is turned on
179     pinMode(GPIOA,2,2); // alternate function mode for PA2
180     selectAlternateFunction(GPIOA,2,7); // AF7 = USART2 TX
181     pinMode(GPIOA,15,2);
182     selectAlternateFunction(GPIOA,15,3);
183     RCC->APB1ENR1 |= (1 << 17); // turn on USART2
184
185     const uint32_t CLOCK_SPEED=80000000;
186     uint32_t BaudRateDivisor;
187
188     BaudRateDivisor = CLOCK_SPEED/baudrate;
189     USART2->CR1 = 0;
190     USART2->CR2 = 0;
191     USART2->CR3 = (1 << 12); // disable over-run errors
192     USART2->BRR = BaudRateDivisor;
193     USART2->CR1 = (1 << 3); // enable the transmitter and receiver
194     USART2->CR1 |= (1 << 2); // enable the transmitter and receiver
195     USART2->CR1 |= (1 << 0);
196 }

```

Figure 18: Function of initializing serial communication and pin definition.

```

197 int _write(int file, char *data, int len)
198 {
199     if ((file != STDOUT_FILENO) && (file != STDERR_FILENO))
200     {
201         errno = EBADF;
202         return -1;
203     }
204     while(len--)
205     {
206         eputc(*data);
207         data++;
208     }
209     return 0;
210 }

```

Figure 19: Function for writing via serial communication.

```

211 void eputc(char c)
212 {
213     while( (USART2->ISR & (1 << 6))!=0); // wait for ongoing transmission to finish
214     USART2->TDR=c;
215 }

```

Figure 20: Function for write via serial communication to terminal display.

```

217 void EXTI4_IRQHandler()
218 {
219     //PIOB->BSRR = (1 << 3); // set PB3 to turn on LED
220     //GPIOB->IDR |= (1 << 4);
221     EXTI->PR1 = (1 << 4); // clear interrupt pending flag
222     pulse_count++; //counting the pulse
223 }
224 }

```

Figure 21: Function to handle interrupt to counting the speed pulse.

```

225 void SysTick_Handler(void)
226 {
227     milliseconds++;
228 }
229 void delay_ms(unsigned dly){
230     unsigned end = milliseconds + dly;
231     while(milliseconds != end);
232 }

```

Figure 22: Function for milliseconds counting and delays.

Testing:

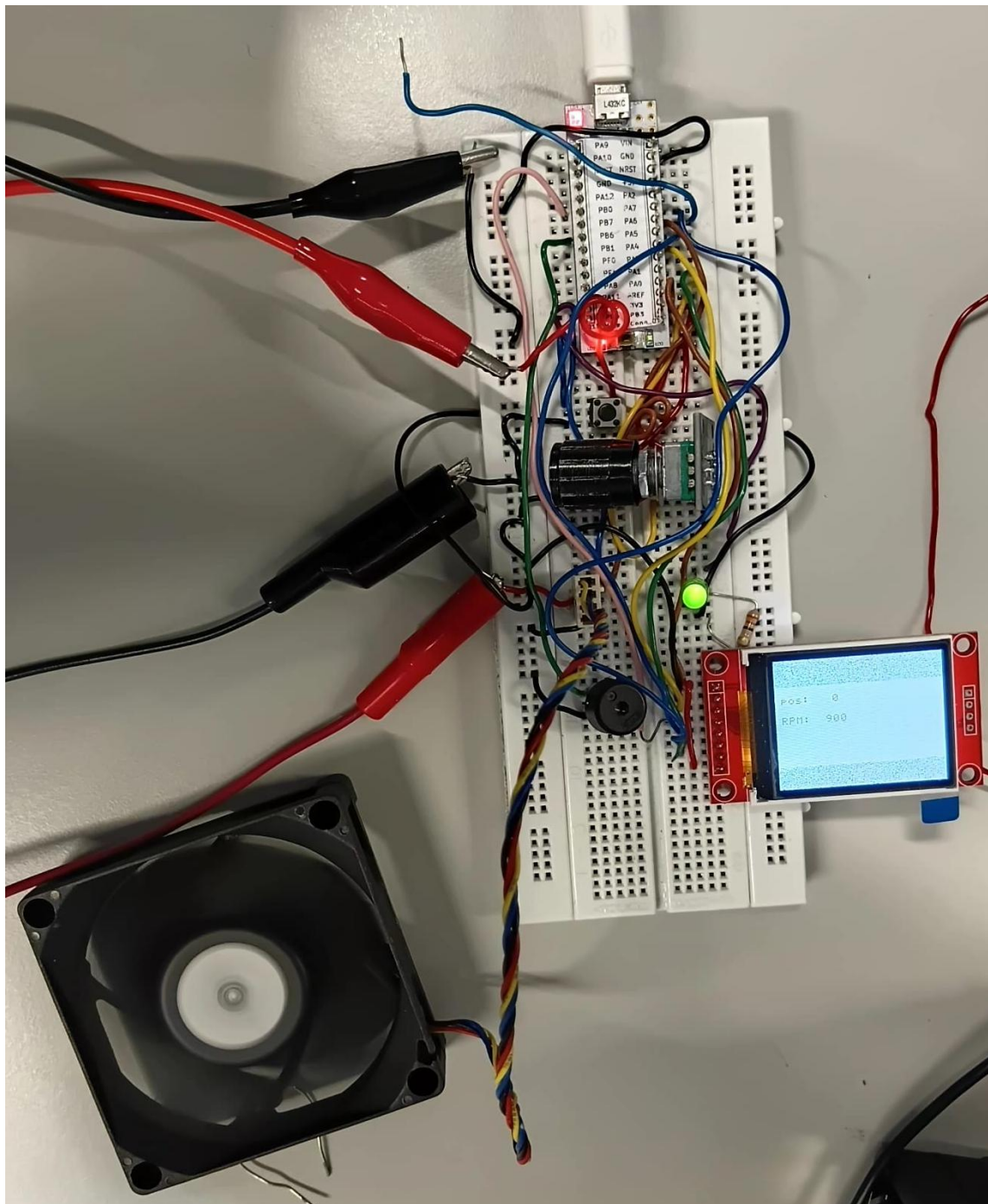


Figure 23: Hardware implementation of bread board.

The whole assemble is shown above in Figure 23 where the PC fan controller is being power,programmed and debugged.

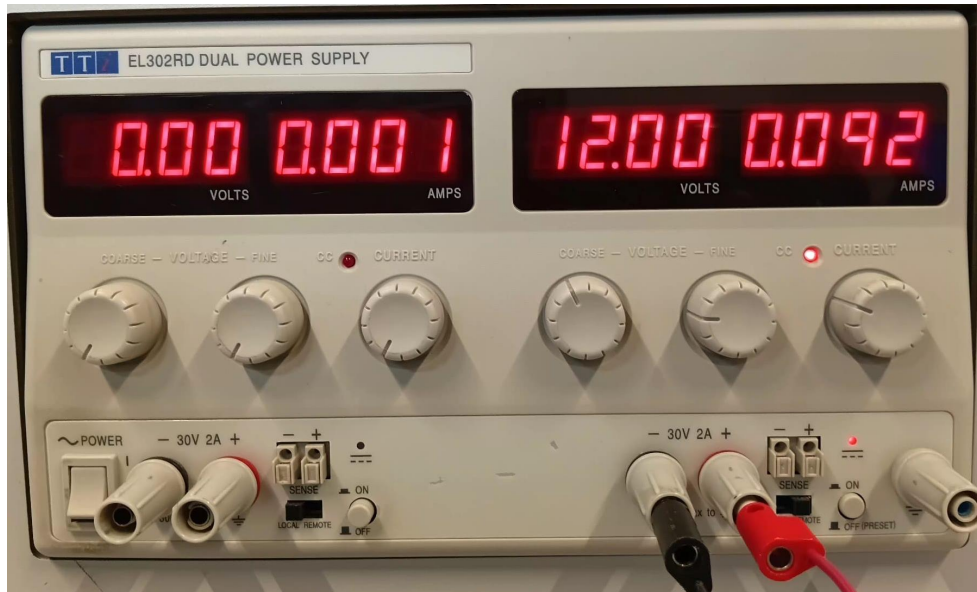


Figure 24: Power supply.

```

--- Quit: Ctrl+C | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H
MODER PA6: 2
AFR PA6: 14
TIM16 CR1: 129
TIM16 CCER: 1
TIM16 BDTR: 32768
TIM16 ARR: 3199
TIM16 CCR1: 1600

```

Figure 25: Output result 1 on terminal.

The printed diagnostic values shown in Figure 25 confirm that Timer 16 and its associated GPIO pin PA6 have been configured correctly for PWM output. PA6's mode register reads 2, indicating alternate function mode, and its alternate function register reads 14, which correctly maps to TIM16 channel 1. Timer 16's control register CR1 shows 129 (binary 10000001), meaning the counter is enabled with auto-reload preload activated. The capture/compare enable register CCER reads 1, confirming the output channel is enabled, while the break and dead-time register BDTR reads 32768 (binary 1000000000000000), indicating the main output enable bit is set—a critical requirement for PWM output on advanced timers. Finally, the auto-reload register ARR contains 3199, producing a 25 kHz PWM signal from the 80 MHz clock, and the capture/compare register CCR1 contains 1600, giving a precise 50% duty cycle. All values match the expected configuration, confirming TIM16 is fully operational and ready to generate PWM signals on pin PA6.

```

ENC: 0, CCR1: 0, RPM: 900, buzz: 0
ENC: 0, CCR1: 0, RPM: 900, buzz: 0
ENC: 0, CCR1: 0, RPM: 900, buzz: 0
ENC: 0, CCR1: 0, RPM: 900, buzz: 0
ENC: 0, CCR1: 0, RPM: 900, buzz: 0
ENC: 0, CCR1: 0, RPM: 900, buzz: 0
ENC: 0, CCR1: 0, RPM: 900, buzz: 0
ENC: 0, CCR1: 0, RPM: 900, buzz: 0
ENC: 0, CCR1: 0, RPM: 900, buzz: 0
ENC: 0, CCR1: 0, RPM: 900, buzz: 0

```

Figure 26: Output result 2 on terminal.

The result in Figure 26 shows the speed of max of 900 revolution per minute when the potentiometer is at minimum level.

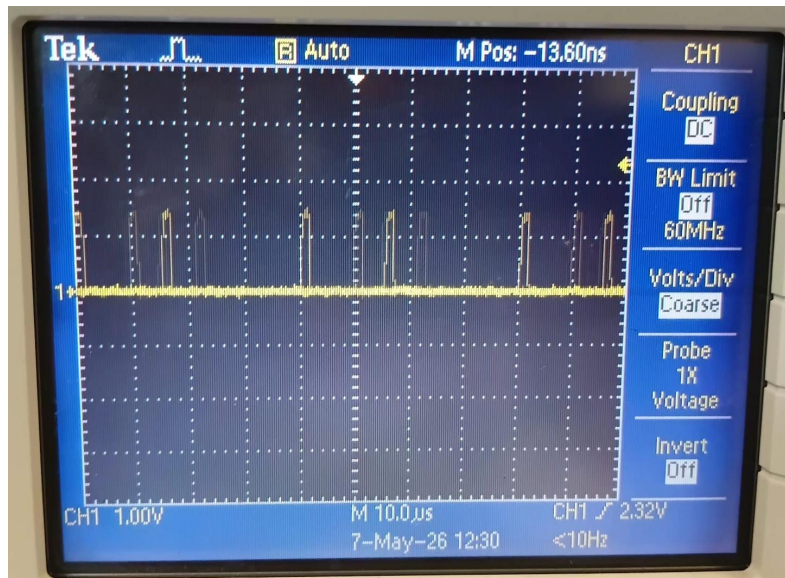


Figure 27: Fan output speed signal 1 viewed on oscilloscope.

The Figure 27 shows the duty cycle at 900 RPM which is almost 0% duty cycle.

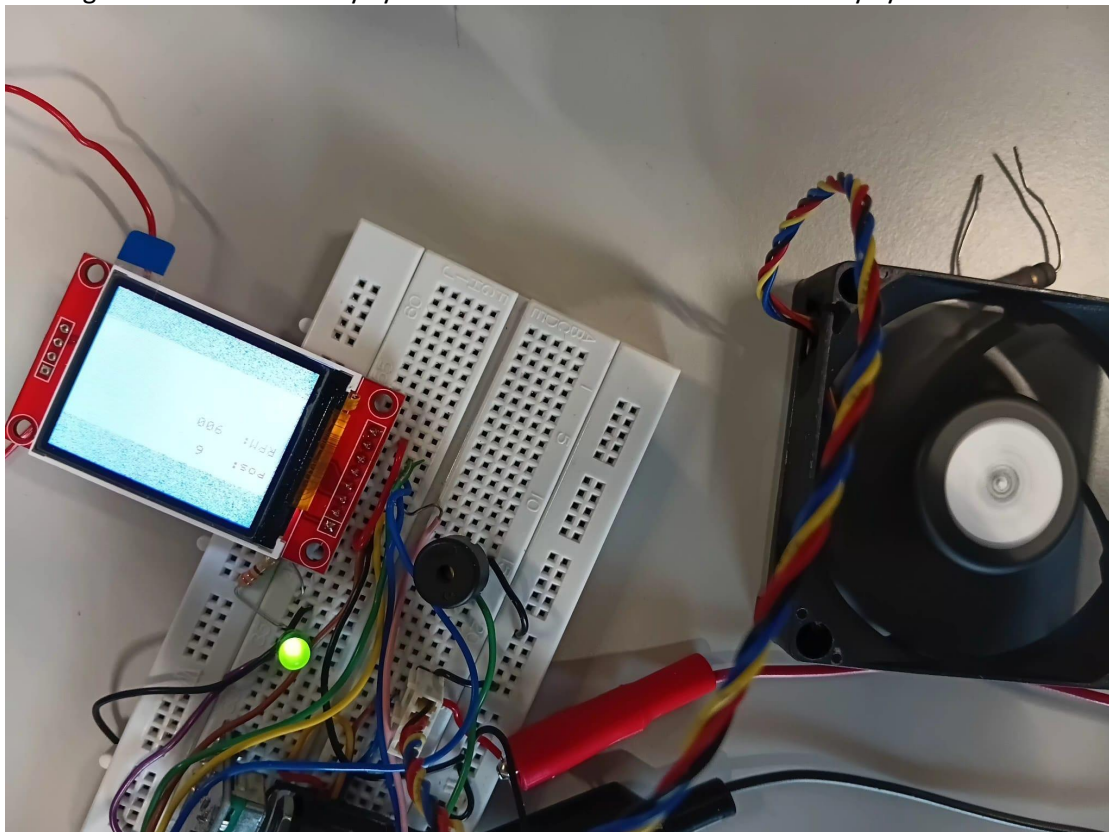


Figure 28: PC fan Assemble with LED ON and Screen .

The result in Figure 28 shows the output speed signal from the PC fan with LED ON, Fan ON and Screen indicating rotary position: 6 and speed 900 RPM.

```
ENC: 149, CCR1: 2997, RPM: 4050, buzz: 0
ENC: 149, CCR1: 2997, RPM: 3600, buzz: 1
ENC: 149, CCR1: 2997, RPM: 4050, buzz: 0
ENC: 149, CCR1: 2997, RPM: 4050, buzz: 1
ENC: 149, CCR1: 2997, RPM: 3600, buzz: 0
ENC: 149, CCR1: 2997, RPM: 3600, buzz: 1
ENC: 149, CCR1: 2997, RPM: 4050, buzz: 0
ENC: 149, CCR1: 2997, RPM: 3600, buzz: 1
ENC: 149, CCR1: 2997, RPM: 4050, buzz: 0
ENC: 149, CCR1: 2997, RPM: 4050, buzz: 1
ENC: 149, CCR1: 2997, RPM: 4050, buzz: 0
```

Figure 29: Output result 2 on terminal.

The result in Figure 29 shows rotary position:149 with speed minimum of 3600 and buzzer toggling between 0 and 1.

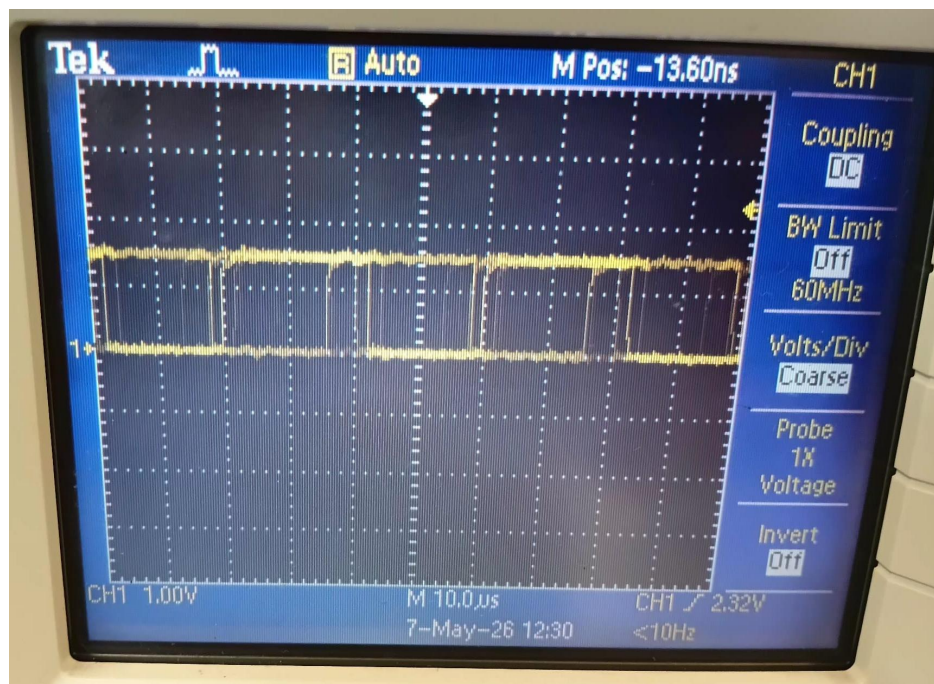


Figure 30: Fan output speed signal 2 viewed on oscilloscope.

The Figure 30 shows the duty cycle at minimum 3600 RPM which is almost 100% duty cycle.

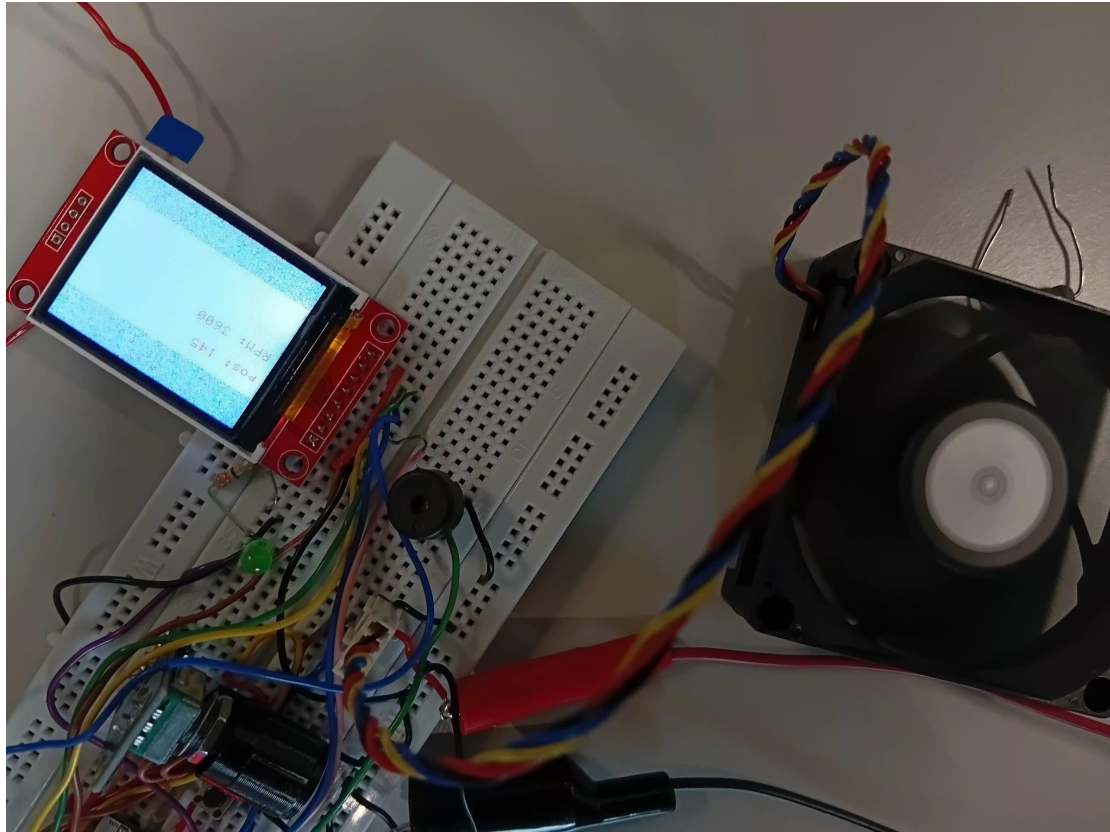


Figure 31: PC fan Assemble with LED OFF, Buzzer toggling and Screen .

The result in Figure 31 shows the output speed signal from the PC fan with LED OFF, Fan ON and at maximum and Screen indicating rotary position: 145 and speed 3600 RPM.

Conclusion:

The Advanced PC Fan Speed Controller using the STM32L432KC microcontroller was successfully designed, implemented, and tested. The project achieved all objectives, including schematic design, software and hardware implementation, testing, debugging, and documentation. The system effectively controlled fan speed through PWM generation, monitored RPM using tachometer feedback, and provided real-time user interaction through the LCD display and serial communication. Safety features such as LED indication and buzzer alerts operated correctly during over speed conditions. Experimental results confirmed accurate PWM control, reliable speed measurement, and stable system performance, demonstrating the effectiveness of the STM32-based embedded control system for PC fan applications.

References:

- [1] [STMicroelectronics STM32L432KC Datasheet](#), “STM32L432KC Datasheet,” STMicroelectronics, Geneva, Switzerland, 2023. Accessed: May 8, 2026.
- [2] [STM32L4 Series Reference Manual RM0394](#), “STM32L4x1 Advanced ARM®-based 32-bit MCUs Reference Manual,” STMicroelectronics, Geneva, Switzerland, 2023. Accessed: May 8, 2026.
- [3] [Omron Tactile Switches](#), “Tactile Switch Specifications,” Omron Electronics, Kyoto, Japan. Accessed: May 8, 2026.
- [4] Kingbright LED Products, “LED Component Datasheets,” Kingbright Electronic Co., Taipei, Taiwan. Accessed: May 8, 2026.
- [5] Vishay Resistors, “Resistor Products and Specifications,” Vishay Intertechnology, Malvern, PA, USA. Accessed: May 8, 2026.
- [6] Bourns Rotary Encoders, “Incremental Rotary Encoder Datasheets,” Bourns Inc., Riverside, CA, USA. Accessed: May 8, 2026.
- [7] [ST7735 TFT LCD Controller Datasheet](#), “ST7735 TFT LCD Controller/Driver Datasheet,” Sitronix Technology Corp., Hsinchu, Taiwan. Accessed: May 8, 2026.
- [8] [Murata Piezoelectric Sound Components](#), “Piezoelectric Sound Components,” Murata Manufacturing Co., Kyoto, Japan. Accessed: May 8, 2026.
- [9] [Intel 4-Wire PWM Controlled Fan Specification](#), “4-Wire Pulse Width Modulation (PWM) Controlled Fans Specification,” Intel Corporation, Santa Clara, CA, USA, Sep. 2005. Accessed: May 8, 2026.
- [10] STMicroelectronics USART Application Note, “USART Communication Application Note,” STMicroelectronics, Geneva, Switzerland. Accessed: May 8, 2026.
- [11] [SparkFun Breadboard Tutorial](#), “How to Use a Breadboard,” SparkFun Electronics, Boulder, CO, USA. Accessed: May 8, 2026.
- [12] Pololu Jumper Wires, “Jumper Wire Specifications,” Pololu Corporation, Las Vegas, NV, USA. Accessed: May 8, 2026.
- [13] Keysight Bench Power Supply Basics, “Fundamentals of DC Power Supplies,” Keysight Technologies, Santa Rosa, CA, USA. Accessed: May 8, 2026.